Predicate Formulation Incompatibility: FakeSciencePredicate Conflict Analysis

Formalization verification

November 13, 2026

Abstract

This note documents the incompatibility between the Classical Mechanics predicate formulation and a conflicting axiomatization supplied by the FakeSciencePredicate module. Using Lean's module system as a verification framework, we demonstrate that the predicate approach provides automatic detection of axiomatizations that cannot coexist. The conflict arises not from subtle semantic disagreement but from explicit redeclaration, making the incompatibility decidable and machine-checkable.

1 Setup: Two Competing Predicates

The foundational Classical Mechanics predicate defines:

$$\mathrm{CM}(\mathcal{M}) \iff P_1 \wedge P_2 \wedge \cdots \wedge P_6,$$

where P_6 encodes Newton's second law:

$$\sum_{i \in I} f(p,t,i) = m(p)\,\frac{\partial^2 s}{\partial t^2}(p,t).$$

The FakeSciencePredicate module purports to also define a predicate on the same namespace (`ClassicalMechanicsPredicate`), but it does so by asserting a false axiom:

```
axiom conflict : False
```

This immediately makes any predicate definition derived from FakeSciencePredicate inconsistent.

2 Conflict Detection Mechanism

2.1 Redeclaration Conflict

When Lean attempts to process both modules in sequence, it encounters an error:

```
environment already contains
'ClassicalMechanicsPredicate.isClassicalMechanicsModel'
from ClassicalMechanicsPredicate
```

This indicates that FakeSciencePredicate cannot redefine an identifier that already exists in the shared namespace.

2.2 False Axiom Dependency

Any attempt to derive theorems from FakeSciencePredicate results in a dependency on the false axiom. Lean's axiom tracking feature reveals:

```
'ClassicalMechanicsPredicate.isClassicalMechanicsModel'
depends on axioms: [ClassicalMechanicsPredicate.conflict]
```

This traces back to the source of inconsistency.

2.3 Proof by Contradiction

If one attempts to combine both definitions within a single file or import context, the type checker fails:

```
'ClassicalMechanicsPredicate.isClassicalMechanicsModel'
has already been declared
```

Lean refuses to proceed, treating the conflict as a compilation error rather than a deferred consistency issue.

3 Why This Matters: The Role of the Predicate

In the Suppes tradition, a set-theoretic predicate is intended to be a *definition* that characterizes a class of models. The predicate succeeds when:

1. It is internally consistent (provable theorems do not depend on false axioms).

2. It correctly captures the intended theory (all axioms are present and sufficient).
3. It does not admit vacuous models (systems satisfying the predicate are genuinely classical mechanical).

FakeSciencePredicate violates all three criteria:

1. It introduces a false axiom, making the namespace itself inconsistent.
2. It attempts to redefine the predicate, corrupting the original definition.
3. Any system claimed to satisfy the predicate would satisfy a false statement, making the class of models empty (in a consistent fragment).

4 Computational Verification

The Lean module system provides automated verification of incompatibility:

1. **Compile-time detection**: Lean's parser and type checker immediately flag the conflict, providing the exact line and error.
2. **No hidden assumptions**: The incompatibility is not buried in subtle proof steps; it is explicit in the module structure.
3. **Transitive inconsistency**: Any module that imports both `ClassicalMechanicsPredicate` and `FakeSciencePredicate` becomes inconsistent.

The audit module `PredicateAudit.lean` formalizes this verification by attempting to reference both definitions and tracing their axiom dependencies.

5 Lessons for Scientific Formalization

5.1 Transparency

The predicate formulation makes the structure of a scientific theory explicit: the choice of axioms, their logical dependencies, and the resulting model class are all visible in the Lean source.

5.2 Decidability

Incompatibility between rival theories is not a matter of opinion or interpretation. The Lean type system decides, automatically and unambiguously, whether two axiomatizations can coexist.

5.3 Modularity

The predicate approach allows theories to be developed independently and then compared. If they are incompatible, that fact is discovered at the moment of composition, not after extensive proof development.

6 Conclusion

The Classical Mechanics predicate formulation, embodied in the explicit conjunction $P_1 \wedge P_2 \wedge \cdots \wedge P_6$, provides a foundation for unambiguous scientific theory comparison. When FakeSciencePredicate introduces a conflicting axiomatization, the Lean system detects the incompatibility deterministically, supporting the view that scientific predicates are not merely informal descriptions but formal, machine-checkable structures.

References